ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ХАРИЛЦАН ШҮҮМЖ

Үйлдлийн систем /F.CSM302/

Сэдэв: “Fork-Join Sorting Application. Programming Project 3 (4-р бүлэг), Recursion tree”

Хуваарилалтын код: IV-2-7

Шүүмжилсэн баг: IV-2-10 — The Sleeping Teaching Assistant

Хичээл заасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн баг: О.Энх /B241940127/
Д.Дагиймаа /B200910032/
Д.Оюунжаргал /B221940034/
Г.Гантогтох /B231930001/
Г.Мягмарсүрэн /B241940014/
Э.Бүрэнбаяр /B242270084/

Огноо: 2026 оны 10 дугаар сарын 26

Гарчиг

1 Ажлын ойлголт **2**

2 Туршилтын орчин **2**

3 Build ба Run **3**

4 Туршилт **3**

4.1 Үндсэн тест (тайланд бичсэнийг давтах) 3

4.2 Нэмэлт тест (өөрсдөө сонгосон) 3

5 Дүрслэлийн үнэлгээ **4**

6 Давуу тал **4**

7 Асуудал **6**

8 Сайжруулах санал **6**

9 Дүгнэлт **7**

1 Ажлын ойлголт

Бодлогын мөн чанар. Sleeping Teaching Assistant нь сонгодог Sleeping Barber бодлогын хувилбар. Нэг туслах багш (TA) оффисдоо суудаг: оюутан байхгүй бол *унтана*, оюутан ирэхэд *сэрч* тусална. TA завгүй үед ирсэн оюутан коридорын хязгаарлагдмал тооны сандлын нэгэнд сууж хүлээнэ. Сандал бүгд дүүрэн бол оюутан буцаж явж, хэсэг хугацааны дараа дахин ирнэ.

Синхрончлолын асуудал. Энэ бодлого нь зэрэгцээ ажиллагааны гурван сонгодог бэрхшээлийг нэг дор агуулдаг:

1. **Алдагдсан сэрээлт (lost wakeup).** Оюутан “TA унтаж байна уу?” гэж шалгаад, сэрээх хүртэлх зай завсарт TA өөр оюутныг авч эхэлж болно. Энэ бол сонгодог *check-then-act* уралдаан.
2. **Хуваалцсан дарааллын бүрэн бүтэн байдал.** Хүлээлгийн дараалал, сандлын тоолуурыг олон thread зэрэг өөрчилдөг тул критик муж шаардлагатай.
3. **Хаалтын дараалал.** Загварыг зогсооход унтаж буй TA болон хүлээж буй оюутнуудыг бүгдийг сэрээж гаргахгүй бол `pthread_join` мөнхөд гацна.

Багийн шийдэл. Хөдөлгүүрийг **C / POSIX Pthreads** дээр бичиж, удирдах цонхыг Python Tkinter дээр хийсэн; хоёулаа `ctypes`-ээр холбогдоно. Оюутан бүр болон TA нь *бодит OS thread*. Хуваалцсан бүх төлөвийг нэг `pthread_mutex_t mu` хамгаална. TA оюутангүй үед `work` нэртэй semaphore дээр *бодитоор блоклогдож* унтдаг — энэ нь “унтах”-ыг давталтаар дуурайгаагүй гэдгийг харуулна. Нэмж `ta_tick`, `ack`, `started` болон оюутан тус бүрийн `ticks[i]`, `done[i]` semaphore-уудаар нэг *tick* доторх үйлдлүүдийг тодорхой дараалалд оруулсан. Иймээс ижил seed, ижил тохиргоо **байт байтаараа ижил** түүх өгдөг — бид үүнийг тусад нь шалгасан (4.2, тест 2).

2 Туршилтын орчин

Үзүүлэлт	Утга
OS	Ubuntu 24.04.5 LTS (Linux 6.18)
Architecture	x86_64
Compiler / runtime	gcc 13.3.0, Python 3.13.16, glibc 2.39
Хувилбар	commit `ec353b0` (`main`), tag байхгүй
Dependency	зөвхөн стандарт libc + pthread; Python pip сан **шаардлагагүй**
Цөмийн тоо	2

Тэмдэглэл: энэ орчинд `python3-tk` *суугаагүй байсан тул график интерфейсийг (Queue animation) энэ машин дээр ажиллуулж чадаагүй. 6-р бүлгийн дүрслэлийн үнэлгээг Linux desktop буюу WSLg орчинд тусад нь хийсэн — тэр хэсгийг багийн гишүүн биечлэн бөглөнө.*

3 Build ба Run

Алхам	Команд	Хүлээгдсэн үр дүн	Бодит үр дүн
Clone	`git clone https://github.com/mendamgalan/IV-2-10-sleeping-ta`	татагдана	**ЗӨВ** — 23 файл, 2.9 MB
Build	`make -C sleeping-ta`	`libta.so` үүснэ	**ЗӨВ** — `-Wall -Wextra -Wpedantic -Werror` дээр **нэг ч анхааруулгагүй**
Test	`make -C sleeping-ta test`	бүх тест давна	**ХЭСЭГЧИЛСЭН** — 14 хөдөлгүүрийн тест давсан, UI тестийн модуль ачаалагдаагүй (6.1-ийг үзнэ үү)
Run	`make -C sleeping-ta run`	цонх нээгдэнэ	**ШАЛГААГҮЙ** — энэ орчинд дэлгэц байхгүй

Manual-аас гадуур хийх шаардлагатай болсон өөрчлөлт: *Байхгүй.* README-ийн алхмууд бичсэнээрээ ажилласан. Энэ нь сайн тал — доорх 7.1-ийг үзнэ үү.

4 Туршилт

4.1 Үндсэн тест (тайланд бичсэнийг давтах)

#	Ороолт / тохиргоо	Хүлээгдсэн үр дүн	Бодит үр дүн
1	`make test` (өөрийн тестийн багц)	18 тест, UI-ийнх нь SKIP	14 давсан, 4 UI тест **ачаалагдсангүй** (ImportError)
2	`resource_failure` — `pthread_create` / `sem_init` алдаа оруулах	`sim_init` нь −3 буцааж цэвэрлэнэ	**ЗӨВ** — 3 нөхцөл бүгд амжилттай
3	Сандал = 0 (`test_zero_chairs`)	хэн ч хүлээхгүй, шууд буцна	**ЗӨВ** — бидний бие даасан ажиллуулалтаар дараалал үргэлж хоосон, 228 татгалзал, 40 үйлчилгээ

4.2 Нэмэлт тест (өөрсдөө сонгосон)

Доорх тестүүдийг багийн тестийн багцаас гадуур, тусдаа бичсэн `ctypes` *хөтлөгчөөр шууд* `libta.so` *дээр ажиллуулсан.*

#	Тохиргоо	Яагаад сонгосон	Хүлээгдсэн	Бодит
1	40 оюутан, 20 сандал, 300 tick	Дээд ачаалал — дараалал халих, буфер хэтрэх эсэх	халихгүй	**ЗӨВ** — 3813 ирэлт, 3593 татгалзал, snapshot 1340 байт (хязгаар 8192)
2	Ижил seed-ээр хоёр удаа	Давтагдах чанарыг баталгаажуулах	ижил түүх	**ЗӨВ** — журнал **байт байтаараа ижил**
3	20 оюутан, 5 сандал, 500 tick — 7671 үйл явдлыг шалгах	Инвариант: хүлээгчийн тоо сандлын тооноос хэтрэх ёсгүй	0 зөрчил	**ЗӨВ** — 0 зөрчил
4	6 хүчингүй оролт (оюутан = 0, = 41; сандал = 21; `amin`= 0; `min`>`max`)	Захын ба хүчингүй утгын боловсруулалт	бүгд татгалзана	**ЗӨВ** — 6/6 нь C түвшинд татгалзсан
5	**ThreadSanitizer** — 3 ажиллуулалт × 200 tick, зэрэг `sim_snapshot()` дуудлагатай	Хуваалцсан төлөв дээр уралдаан (data race) байгаа эсэх	0 race	**ЗӨВ — нэг ч race илрээгүй**
6	**AddressSanitizer** + **UBSan**, мөн **Valgrind** `-leak-check=full`	Санах ойн алдаа, алдагдал	0 алдаа	**ЗӨВ** — алдаа, алдагдал **байхгүй**

5 Дүрслэлийн үнэлгээ

Багшийн оноосон дүрслэл: ***Queue animation****. Доорхийг график интерфейсийг биечлэн ажиллуулж бөглөв.*

Шалгах асуулт	Тийм/Үгүй	Нотолгоо
Ролт, тохиргоог өөрчлөхөд дүрслэл шинэчлэгдэж байна уу?		
Дараалал дахь оюутны тоо бодит төлөвтэй нийцэж байна уу?		
ТА-ийн төлөв (унтаж / тусалж) зөв харагдаж байна уу?		
Тэмдэглэгээ, legend, нэгж ойлгомжтой юу?		
Алхамчилж харах эсвэл эргэн шалгах боломжтой юу?		
Дүрслэл нь алгоритмыг тайлбарлах *нотолгоо* болж байна уу?		

Дүгнэлт:

6 Давуу тал

1. ТА нь *бодитоор* блоклогддог — “унтах”-ыг дуурайгаагүй.

Нотолгоо: `src/engine.c:81` — `wait_sem(&work)`. Оюутан байхгүй үед ТА-ийн thread нь semaphore дээр зогсоно, завсарлагатай давталтаар (busy-wait) шалгаагүй. Энэ нь сурах бичгийн

бодлогын мөн чанарыг зөв илэрхийлж байна. Мөн `wait_sem` нь `EINTR`-ийг зөв боловсруулдаг (`engine.c:40-50`) — энэ нарийн зүйлийг оюутны ажилд ховор хардаг.

2. Алдаа оруулах (fault injection) тесттэй.

Нотолгоо: `tests/resource_failure.c` нь макроор `pthread_create` болон `sem_init`-ийг орлуулж, 1-р thread, 4-р thread, 6-р semaphore дээр зориудаар алдаа үүсгэнэ. Ингэснээр *хэсэгчилсэн эхлэлийн цэвэрлэгээ* зөв эсэхийг шалгадаг — ихэнх багт огт байдаггүй түвшний тест. Бид ажиллуулахад 3/3 нөхцөл амжилттай.

3. Бүрэн бөгөөд үнэнч баримтжуулалт.

Нотолгоо: README нь энэ нь *алхамчилсан загвар* бөгөөд “чөлөөт OS scheduling-ийн race/stress benchmark **биш**” гэдгийг өөрөө ил хэлсэн. Мөн Reset/хаахаас өмнө CSV экспортлох шаардлагыг анхааруулсан. Хязгаарлалтаа нуухгүй бичсэн нь ажлын найдвартай байдлыг нэмэгдүүлж байна. Нэмж GitHub Actions CI нь push бүрд warnings-as-errors build болон Xvfb-тэй UI тест ажиллуулдаг.

7 Асуудал

Priority	Асуудал (давтах алхамтай)	Нөлөө	Сайжруулах санал
Critical	*Илрээгүй.*	—	—
Major	`make test` нь `python3-tk` суугаагүй орчинд SKIP биш **ERROR** болно. Давтах: `python3-tk`-гүй Linux дээр `make -C sleeping-ta test`. Хүлээгдсэн: README-ийн дагуу UI тест SKIP. Бодит: `tests/test_ui.py:3` дахь модулийн түвшний `import tkinter` нь `ImportError` өгч, 14 тест давсан хэдий ч `make` нь `Error 2` буцаана.	Шүүмжлэгч эсвэл багш “тест унасан” гэж буруу дүгнэх эрсдэлтэй. Дэлгэцгүй сервер дээр CI хийх боломжгүй.	`test_ui.py`-д импортыг хамгаалах: хамгаалсан `try / except ImportError` хэлбэрт оруулж `unittest.SkipTest` өгөх (8.1-ийг үзнэ үү). Ингэснээр одоогийн дэлгэцгүй SKIP-тэй адил ажиллана.
Minor	`sim_recent_events()` ба `sim_events()` нь хуваалцсан төлөвийг (`recent[]`, `recent_count`, `journal`, `journal_error`) `mu`-г **барилгүй** уншина (`engine.c:322-363`). Одоогийн хяналтын урсгал зөвхөн tick-ийн хооронд дууддаг тул аюулгүй — бид ThreadSanitizer-ээр баталсан.	Инвариантыг зөвхөн *дуудагчийн сахилга* барьж байна. Хожим UI-г өөр thread-ээс шинэчилбэл **чимээгүй** race үүснэ.	Эдгээр функцийг ч `mu` дотор оруулах, эсвэл `assert(!in_tick)` маягийн хамгаалалт нэмэх.
Minor	`sim_snapshot()` дотор `snprintf`-ийн буцах утгыг шалгалгүй `p`-д нэмж байна (`engine.c:300-316`). `snprintf` нь таслагдсан үед *бичих байсан* уртыг буцаадаг тул `p` нь буферээс хэтэрч, `sizeof(out) - (size_t)p` нь асар том `size_t` болно.	Одоогийн хязгаараар (40 оюутан → 1340 байт, буфер 8192) **хүрэхгүй** — бид дээд ачаалал дээр хэмжсэн. Гэхдээ `MAX_STUDENTS` нэмэгдвэл чимээгүй буфер халилт болно.	Үлдсэн зайг шалгах: `if (p >= (int)sizeof(out)) ...` эсвэл туслах `append()` функц ашиглах.
Suggestion	Хувилбарын **tag байхгүй** (`git tag` хоосон) ба `git shortlog -sn -all` нь **4** нэр л харуулж байна (B242270008: 12, Mend-Amgalan: 4, Orgilsakhan: 2, manlaibaatarmaggir: 1).	“9 дэх долоо хоногт нийтэлсэн хувилбар” гэдгийг заах боломжгүй. Багийн гишүүн бүрийн хувь нэмэр `git log`-оос бүрэн харагдахгүй байж магадгүй.	`v1.0` tag тавих. Commit хийгээгүй гишүүд өөрийн нэрээр дор хаяж нэг commit хийх, эсвэл хувь нэмрээ README-д тодорхой бичих.

Түвшин	Утга
Critical	Үндсэн функц ажиллахгүй эсвэл буруу үр дүн
Major	Гол тест, дүрслэл, Manual ноцтой дутуу
Minor	Үндсэн ажиллагаанд саад багатай чанарын асуудал
Suggestion	Нэмэлт сайжруулалтын санал

8 Сайжруулах санал

1. `tests/test_ui.py`-ийн 3-р мөрөнд байгаа `import tkinter as tk` нь `python3-tk` суугаагүй орчинд бүх тестийн багцыг унагааж байна. Үүнийг хамгаалсан импорт болгох саналтай:

```
try:
    import tkinter as tk
except ImportError:
    raise unittest.SkipTest('tkinter (python3-tk) суураагүй')
```

Ингэснээр одоогийн "дэлгэц байхгүй" SKIP-тэй ижил зан гаргаж, README-д бичсэн "UI-ийн 4 тест SKIP болно" гэсэн амлалт бүх орчинд үнэн болно.

2. `sim_snapshot()` дотор `snprintf`-ийн үр дүнг хуримтлуулахын өмнө таслагдалтыг шалгах. Одоо хүрэхгүй ч, `MAX_STUDENTS`-ийг 40-өөс дээш болговол чимээгүй буфер халилт болно. Хамгийн энгийн шийдэл нь үлдсэн зайг шалгадаг жижиг `append()` туслах функц бичих.

3. `v1.0` tag тавьж, README-д багийн гишүүн бүрийн хариуцсан ажлын хүснэгт нэмэх. Одоо `git shortlog` 4 нэр харуулж байгаа тул багш гишүүн бүрийн оролцоог шалгахад хүндрэлтэй. Хүснэгт нь `git`-ийн түүхтэй хамт бүрэн зураг өгнө.

9 Дүгнэлт

1. `test_ui.py`-ийн импортыг хамгаалж, `make test` нь `python3-tk`-гүй орчинд ч бүрэн давдаг болгох. *(Хамгийн хялбар, хамгийн их нөлөөтэй засвар — 3 мөр.)*

2. `sim_recent_events()` / `sim_events()`-ийн түгжээний гэрээг кодод илэрхий болгох (түгжээ дотор оруулах эсвэл assert нэмэх), мөн `sim_snapshot()`-ийн `snprintf` таслагдалтыг шалгах.

3. `v1.0` tag тавьж, багийн гишүүдийн хариуцсан ажлыг README-д баримтжуулах.

Ерөнхий үнэлгээ: Энэ бол **маш сайн бэлтгэгдсэн** ажил. Хөдөлгүүр нь бодит POSIX thread, semaphore, mutex ашигласан; TA нь semaphore дээр бодитоор блоклогддог; хуваалцсан төлөв бүхэлдээ нэг түгжээгээр хамгаалагдсан. Бидний бие даасан шалгалтаар **ThreadSanitizer, AddressSanitizer, UBSan, Valgrind дөрвүүлээ цэвэр**; ижил seed дээр үр дүн байт бай-тараа давтагдана; 500 tick, 7671 үйл явдал дотор "хүлээгч $\leq$ сандал" инвариант нэг ч удаа зөрчигдөөгүй; хүчингүй оролт 6/6 татгалзагдсан. Компайл нь `-Werror` дээр нэг ч анхааруулгагүй.

Илэрсэн цорын ганц ноцтой асуудал нь кодын биш, **тестийн багцын** асуудал — нэмэлт сангүй орчинд SKIP хийх ёстой UI тест нь бүх багцыг унагааж байна. Үлдсэн хоёр нь одоогоор хүрэхгүй боловч ирээдүйд аюултай болж болох *далд* эрсдэлүүд. Эдгээрийг засвал ажил бүрэн гүйцэд болно.